boxes, during the design of an application or algorithm using them.

- **Implementation**: when we move from abstract data structures to their concrete implementations in a programming language, we need to pay attention to those nitty-gritty details that are specific to the language we chose, and make sure we don't slow down our methods by choosing the wrong implementation.
- **Tree-array duality**: a heap is conceptually a tree, but it's implemented using an array for the sake of efficiency. This is only possible because a d-ary heap is not just any tree, but a complete, left-adjusted tree.
- **Branching factor**: Changing the branching factor of a heap won't affect asymptotic running time for its methods, but will still provide a constant factor improvement, that matters when we move from pure theory to applications with a massive amount of data to manage.
- **Continuous improvement**: even small changes can make a great difference, and you need to challenge your assumptions in order to find better solutions: we could have stopped at binary heaps after all. You can see this all over the place in algorithms literature: from 2-3 trees to soft heaps.
- **Applications**: for several advanced algorithms we make use of priority queues to improve their performance: BFS, Dijkstra and Huffman codes are just some examples.
- **Profiling**: when high-performance is paramount, and the data structures we use allow parameters (like *branching factor* for heaps), the only way to find the best parameters for our implementation is profiling our